M05 Learner Book

Python for Data Engineering

Release Candidate 1 - Data Engineering Python production candidate

Scope rule

Python remains the programming foundation. Pandas is one local transformation tool; APIs, validation, logging, files, database loading and reliable failure behavior matter just as much.

M05-L01 - CSV Properly

Learning objective

Read/write CSV while respecting headers, delimiters, quoting, missing values and raw-source preservation.

Estimated focused time: 3-4 hours including G01

CSV looks simple because you can open it in a text editor, but commas may appear inside quoted fields, files may use different delimiters, and all values initially arrive as text. Use a CSV parser rather than splitting every line on commas.

```
import csv
with open("orders.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["order_id"], row["amount"])
```

Missing is not zero

An empty amount, "0", and missing column are different states. Preserve the raw value long enough to decide whether the business rule allows conversion/defaulting.

Writing output

```
with open("clean.csv","w",newline="",encoding="utf-8") as f:
    writer=csv.DictWriter(f,fieldnames=["order_id","amount"])
    writer.writeheader(); writer.writerows(clean_rows)
```

Common mistakes and debugging

- Using line.split(",") for general CSV.
- Overwriting the raw input with cleaned output.
- Treating empty strings as numeric zero without a rule.

Why this matters in Data Engineering

CSV remains common for file drops, exports, partner feeds and small batch interfaces. Reliable ingestion starts with preserving raw evidence.

Your turn - do this before checking review

Read orders.csv with DictReader. Count raw rows, blank amounts and repeated order IDs. Do not clean yet.

Check yourself

[ ] I use a CSV parser.

[ ] I distinguish missing from zero.

[ ] I preserve source data.

M05-L02 - JSON and JSONL

Learning objective

Parse nested JSON and line-delimited JSON while understanding object/list structure.

Estimated focused time: 3-4 hours including G02

JSON maps naturally to Python dictionaries, lists, strings, numbers, booleans and null/None. A JSON document may contain one nested object, while JSON Lines (JSONL/NDJSON) stores one JSON object per line and is convenient for streaming/batch processing.

```
import json
data=json.loads(text)
for customer in data["customers"]:
    print(customer["customer_id"])

for line in Path("events.jsonl").read_text().splitlines():
    event=json.loads(line)
```

Do not assume nested keys always exist. First inspect shape and define required vs optional fields.

Common mistakes and debugging

- Confusing a list of objects with one object.
- Using eval instead of json parsing.
- Crashing on an optional nested key without deciding a fallback/validation rule.

Your turn - do this before checking review

Read customers.json and build a dictionary mapping customer_id -> region. Then read events.jsonl and count events per order.

Check yourself

[ ] I know JSON object -> dict and array -> list.

[ ] I can read JSONL one line at a time.

[ ] I inspect shape before indexing deeply.

M05-L03 - Paths, Folders and Batch File Processing

Learning objective

Discover files safely, process a batch deterministically and separate landing/processed/archive thinking.

Estimated focused time: 3-4 hours including G03

```
from pathlib import Path
files=sorted(Path("batch").glob("items_*.csv"))
for path in files:
    print(path.name)
```

Sorting a discovered file list gives deterministic processing order. File names are data too: validate patterns rather than accepting any file in a landing folder.

Do not move before success

A common reliability mistake is archiving/moving the input before processing completes. Track processing state so a failed file can be retried without guessing where it went.

Folder pattern

```
landing/
processing/
archive/
quarantine/
```

Common mistakes and debugging

- Processing every file including temporary/backup files.
- Archiving before successful output/validation.
- Depending on OS directory iteration order.

Your turn - do this before checking review

Discover only items_YYYYMMDD.csv files in the batch folder, print them in date order and count rows in each.

Check yourself

[ ] I can discover files with glob.

[ ] I sort deterministic inputs.

[ ] I understand archive/quarantine are workflow states.

M05-L04 - Pandas Foundations

Learning objective

Use DataFrames for local tabular inspection/filtering without treating Pandas as the whole of Python.

Estimated focused time: 4-5 hours including G04

```
import pandas as pd
df=pd.read_csv("orders.csv")
print(df.shape)
print(df.dtypes)
print(df.head())
```

Before transforming, inspect shape, columns, types, null counts and a sample. A DataFrame is convenient for moderate local data; later Spark handles distributed workloads.

```
subset=df.loc[df["customer_id"]=="C01", ["order_id","amount"]]
```

Copy/assignment discipline

Prefer clear .loc selections/assignments and avoid code whose mutation semantics you cannot explain. Keep raw_df separate from cleaned_df when preserving evidence matters.

Common mistakes and debugging

- Immediately transforming before profiling.
- Assuming inferred dtype is correct.
- Using Pandas for a task that simple SQL/standard Python handles more clearly.

Your turn - do this before checking review

Load orders.csv. Report shape, columns, dtypes, null counts and duplicate order_id count before cleaning.

Check yourself

[ ] I profile before transform.

[ ] I can select rows/columns.

[ ] I know inferred types require validation.

M05-L05 - Cleaning with Pandas

Learning objective

Handle missing values, duplicates, strings, numeric/date conversion and explicit rejection rules.

Estimated focused time: 4-5 hours including G05

```
clean=df.copy()
clean["amount_num"]=pd.to_numeric(clean["amount"],errors="coerce")
clean["order_date"]=pd.to_datetime(clean["order_date"],errors="coerce")
```

errors="coerce" turns invalid conversion into missing values; that does not make them valid. It creates evidence you must inspect and route according to rules.

Duplicate keys

```
dupes=clean[clean.duplicated("order_id",keep=False)]
```

Do not drop duplicates until you know the business key and winner rule. Exact repeated rows and updated versions are different situations.

Common mistakes and debugging

- drop_duplicates() before examining what was dropped.
- fillna(0) across all columns.
- Parsing dates without checking invalid results.

Your turn - do this before checking review

Create valid and rejected DataFrames from orders.csv. Reject missing/non-numeric amount and duplicate order_id; preserve a reason column.

Check yourself

[ ] I inspect duplicates before dropping.

[ ] I treat coercion failures as validation evidence.

[ ] I keep rejected records.

M05-L06 - Transforming with Pandas

Learning objective

Join, group and derive columns while protecting row grain and validating merge cardinality.

Estimated focused time: 4-5 hours including G06

```
merged=orders.merge(customers,on="customer_id",how="left",validate="many_to_one",
indicator=True)
```

validate="many_to_one" is a powerful guard when many orders should map to one customer. If the customer master contains duplicate keys, the merge should fail rather than silently multiplying rows.

```
summary=(merged.groupby("region",dropna=False)["amount_num"].sum().reset_index())
```

Check before/after counts

A left enrichment merge should normally preserve the left row count. Compare counts and inspect _merge to expose unmatched keys.

Common mistakes and debugging

- Merging on the wrong key because names look similar.
- Ignoring row multiplication.
- Grouping before deciding whether invalid/unmatched rows belong.

Why this matters in Data Engineering

Join cardinality reasoning from SQL applies directly to Pandas merges. The tool changes; the data logic does not.

Your turn - do this before checking review

Merge cleaned orders to customers.json-derived DataFrame. Preserve unmatched customers and summarize valid amount by region.

Check yourself

[ ] I can use merge validate.

[ ] I check row count before/after.

[ ] I keep unmatched rows visible.

M05-L07 - Data Validation in Python

Learning objective

Turn business requirements into reusable checks for schema, nulls, types, ranges, uniqueness and relationships.

Estimated focused time: 4-5 hours including G07

Validation answers “Is this data acceptable for this pipeline stage?” A check should be explicit enough that another engineer can understand what failed and how many records were affected.

```
required={"order_id","customer_id","amount"}
missing=required-set(df.columns)
if missing:
    raise ValueError(f"Missing columns: {sorted(missing)}")
```

Record vs batch rules

- Record rule: amount >= 0.
- Record rule: customer_id present.
- Batch rule: order_id unique.
- Batch rule: required columns exist.
- Relationship rule: every customer_id exists in master, unless exceptions are allowed.

Some failures justify rejecting one row; others mean the whole batch is unsafe, such as a missing required column.

Common mistakes and debugging

- One generic “invalid” flag with no reason.
- Continuing after a missing critical column.
- Silently removing invalid rows without counts/reasons.

Your turn - do this before checking review

Write validate_required_columns and duplicate_key_report functions. Test both on a good and intentionally bad DataFrame.

Check yourself

[ ] I distinguish batch vs record failures.

[ ] I return/report reasons.

[ ] I know validation is business-rule driven.

M05-L08 - HTTP and REST APIs from Zero

Learning objective

Understand client/server, URL, endpoint, method, parameters, headers, status code and JSON response before coding.

Estimated focused time: 3-4 hours including G08

An API lets one program request data or actions from another system through a defined interface. Your script is the client; the API service is the server. An endpoint is a particular address/path with a contract.

Concept	Example

Method	GET - retrieve data
Endpoint	/orders
Query parameter	?page=2
Header	X-API-Key: ...
Status	200 success, 404 not found, 429 rate limit, 500 server failure
Body	Often JSON data

A 200 response can still contain bad business data; transport success and data quality are separate layers.

Common mistakes and debugging

- Treating every non-200 response the same.
- Assuming an endpoint returns all records in one response.
- Confusing authentication with authorization/business validity.

Your turn - do this before checking review

With the local API running, visit/request /orders?page=1 and identify endpoint, query parameter, status code, top-level JSON keys and next-page signal.

Check yourself

[ ] I can explain request vs response.

[ ] I know status code is transport/protocol evidence.

[ ] I understand pagination can require multiple requests.

M05-L09 - Calling APIs with Requests

Learning objective

Send HTTP requests with timeouts, parameters and explicit response handling.

Estimated focused time: 4-5 hours including G08

```
import requests
r=requests.get("http://127.0.0.1:8765/orders",params={"page":1},timeout=5)
r.raise_for_status()
data=r.json()
```

Always use a timeout. Without one, a script can wait far longer than intended for a response. raise_for_status converts many HTTP failures into exceptions you can handle intentionally.

Inspect before assuming

- status_code
- response headers when relevant
- JSON/content type
- required fields
- record count

Common mistakes and debugging

- No timeout.
- Calling .json() before checking whether response is successful/JSON.
- Hardcoding URLs repeatedly across functions.

Your turn - do this before checking review

Write fetch_page(page) for the local /orders endpoint. Return the parsed JSON and print item count for page 1.

Check yourself

[ ] I use timeout.

[ ] I check HTTP failure.

[ ] I can pass query parameters cleanly.

M05-L10 - Pagination and Authentication Concepts

Learning objective

Retrieve all pages safely and pass credentials through headers/environment rather than embedding real secrets in code.

Estimated focused time: 4-5 hours including G09

```
page=1
all_items=[]
while page is not None:
    data=fetch_page(page)
    all_items.extend(data["items"])
    page=data["next_page"]
```

Pagination may use page numbers, offsets, cursors or next URLs. Never assume page=1..N unless the API contract says so. Preserve enough state to know what pages succeeded.

Authentication

```
api_key=os.environ["TRAINING_API_KEY"]
headers={"X-API-Key":api_key}
```

Credentials belong outside committed source. The local training key is fake, but the habit should match real secret handling.

Common mistakes and debugging

- Stopping after first page.
- Infinite pagination loop because next token never changes.
- Printing real API keys into logs.

Your turn - do this before checking review

Fetch all /orders pages and confirm the total item count. Then call /secure-products using TRAINING_API_KEY from your shell environment.

Check yourself

[ ] I can loop until next_page is None.

[ ] I keep secrets outside code.

[ ] I avoid logging credentials.

M05-L11 - Reliable API Handling

Learning objective

Classify failures, use limited retries/backoff, respect rate limits and avoid partial-success confusion.

Estimated focused time: 4-5 hours including G10

Retry only failures likely to improve with time: temporary server errors, selected connection errors or rate limits. Invalid credentials or malformed requests usually require repair, not repeated identical requests.

```
for attempt in range(1,4):
    try:
        r=requests.get(url,timeout=2)
        if r.status_code==429:
            time.sleep(1)
            continue
        r.raise_for_status()
        break
    except requests.RequestException:
        if attempt==3: raise
        time.sleep(attempt)
```

Partial extraction

If page 1 and 2 succeed but page 3 fails, do not claim the dataset is complete. Record successful pages/raw responses and mark the run incomplete until recovery logic is defined.

Common mistakes and debugging

- Retrying 401 forever.
- Infinite/unbounded retries.
- Continuing as “success” after one page failed.
- Sleeping blindly instead of respecting Retry-After when supplied.

Why this matters in Data Engineering

Reliable ingestion is not “requests.get worked once.” It is bounded retries, clear failure state, raw evidence and safe recovery.

Your turn - do this before checking review

Call /flaky-orders: first call fails, next succeeds. Implement at most 3 attempts and log each attempt. Then handle /rate-limited once.

Check yourself

[ ] I distinguish retryable from non-retryable.

[ ] I bound retries.

[ ] I do not hide partial failure.

M05-L12 - Python + PostgreSQL

Learning objective

Connect through SQLAlchemy/psycopg, parameterize SQL and load validated records without embedding credentials.

Estimated focused time: 4-5 hours including G11

Python should connect to the database using a connection URL supplied by environment/config. Keep SQL parameterized; do not build queries by string-concatenating untrusted values.

```
from sqlalchemy import create_engine, text
engine=create_engine(os.environ["DATABASE_URL"])
with engine.begin() as conn:
    conn.execute(text("INSERT INTO m05_orders(order_id,amount) VALUES
(:id,:amount)"), {"id":"O1","amount":100})
```

Transactions

engine.begin() commits if the block succeeds and rolls back if an exception escapes. Database constraints remain a second line of defense after Python validation.

Idempotency introduction

A rerunnable load should not double-insert the same business key. In M08 you will study idempotency deeply; here you practice a primary key plus ON CONFLICT/controlled upsert.

Common mistakes and debugging

- Credentials hardcoded in script.
- SQL assembled with f-strings from data values.
- Loading invalid rows first and “cleaning later.”
- Assuming Python validation replaces database constraints.

Your turn - do this before checking review

Create a local m05_test_orders table and insert two validated sample rows using parameterized SQL. Rerun with an upsert/conflict strategy and confirm row count does not double.

Check yourself

[ ] I can use DATABASE_URL.

[ ] I parameterize values.

[ ] I understand transaction and duplicate-load basics.

M05-L13 - Configuration, Secrets and Environment Variables

Learning objective

Separate code, configuration and secrets so the same program can run safely in different environments.

Estimated focused time: 3-4 hours

Configuration changes how a program runs: input path, API base URL, timeout, database URL. A secret is sensitive configuration such as a password/token. Both should be separated from source logic, but secrets require stricter handling.

```
API_BASE_URL=os.getenv("API_BASE_URL","http://127.0.0.1:8765")
DB_URL=os.environ["DATABASE_URL"]
```

.env awareness

Local development often loads environment variables from a .env file, but .env containing secrets must be ignored by Git. Supply a .env.example with variable names/placeholders only.

Common mistakes and debugging

- Committing a real .env.
- Printing configuration dictionaries that contain secrets.
- Hardcoding machine-specific paths and credentials.

Your turn - do this before checking review

Create .env.example containing API_BASE_URL= and DATABASE_URL= placeholders. Set API_BASE_URL in your shell and read it with os.getenv.

Check yourself

[ ] I know config vs secret.

[ ] I keep secrets out of code/Git.

[ ] I can read environment variables.

M05-L14 - Logging, Testing and Reusable Pipeline Code

Learning objective

Replace print-only scripts with structured logging, testable functions and small automated checks.

Estimated focused time: 4-5 hours including G12

```
import logging
logging.basicConfig(level=logging.INFO,format="%(asctime)s %(levelname)s %
(message)s")
logging.info("Starting ingestion")
```

Logs should record what happened without leaking secrets or dumping huge raw datasets. Useful fields include source, run ID, counts, page/file, status and error reason.

Testing small logic

```
def is_valid_amount(value):
    return value >= 0

def test_amount():
    assert is_valid_amount(0)
    assert not is_valid_amount(-1)
```

Tests are easiest when logic is inside small functions rather than mixed with network/database side effects. Separate fetch, validate, transform and load responsibilities.

Suggested project shape

```
src/
  extract.py
  validate.py
  transform.py
  load.py
  main.py
tests/
```

```
logs/
raw/
```

Common mistakes and debugging

- Only print statements and no useful log levels/context.
- Testing by rerunning the full pipeline for every small rule.
- One 500-line script with network, validation and database logic inseparable.

Your turn - do this before checking review

Move one validation rule into a function and write two pytest cases. Add INFO logging around a small extraction function.

Check yourself

[ ] I can log counts/status.

[ ] I can write a tiny pytest test.

[ ] I separate side effects from pure logic where practical.

M05-L15 - Python DE Integration Pipeline

Learning objective

Combine files, API pagination, validation, raw preservation, transformations, logging and database loading into a rerunnable small pipeline.

Estimated focused time: 6-8 hours

The integration pattern is the bridge into M08 ETL/ELT. M05 proves you can implement the mechanics; M08 will deepen pipeline state, watermarks, idempotency, reconciliation and recovery.

```
extract -> save raw -> validate -> transform -> load -> verify -> log summary
```

Run-level evidence

- run start/end
- source page/file counts
- raw copy locations
- valid/rejected counts
- loaded/upserted count
- failure status/reason
- no secret values in logs

Rerun question

Run the exact same input twice. If the target doubles incorrectly, the pipeline is not yet safe. At this stage, primary keys/upsert logic and raw-output naming are enough to introduce the habit.

Your turn - do this before checking review

Complete the independent integration task using the local API and customer file. Save raw pages, rejected rows and run log. Explain what happens when rerun.

Check yourself

[ ] I preserve raw API responses.

[ ] I validate before load.

[ ] I can explain rerun behavior.